Symphony6 NASC3.x Release Manual

Document Number: LZ0000717_01

Revision Number: 0.0.6

Revision Date: Jan. 30, 2026

Revision History

Revision Number	Revision Date	Changes	
		Page Number	Description
0.0.1	Jul.21, 2025	-	Initial Release
0.0.2	Aug.21, 2025	-	Update section 2.2 flash program
0.0.3	Nov.11, 2025	-	Update section 5 and add section 6.
0.0.4	Dec.12, 2025	-	Add SCS image debug and DVB/PVR/OTT operation description
0.0.5	Jan.30, 2026	-	Add description about OTT re-encryption and output control
0.0.6	Fen.13, 2026		Add section 11 Brief flow for NAGRA

Table of Contents

Acronyms 1

Reference 1

2.2 How to use release package 3

2.2.1 Patch it to our DDK 3

2.2.2 Demo project 4

2.3 Flash program 4

2.3.1 Setup hardware environment 4

2.3.2 Flash program 8

2.3.2.1 Flash write command 10

2.3.2.2 Flash read command 10

2.3.2.3 Update flash data by debug Boot 10

3 NOCS3.x chipset introduce 11

4 Bootloader 11

5 TEE related content (NASC3.x Dual Only) 11

5.1 TA files 11

5.2 TEE-OS files 13

6 OTP brief 13

7 Get singed SCS image and program to board. 13

7.1 Get singed SCS image from NAGRA 14

7.2 Program singed SCS image to board 14

7.3 Test security boot of SCS image. 15

8 Brief flow of Montage-LZ case 16

9 HDCP key encryption and loading..... 23

10 Output control 24

11 Brief flow for NAGRA 25

Acronyms

Acronym	Stands for	Definition
CA	Condition Access	NAGRA control it
SCS	Secure Chipset Start-up	First boot image for chipset
APCPU	Application CPU	Application CPU in normal mode to run REE
AVCPU	Audio Video decode CPU	Help to decode audio and video
AuxCode	SCS Auxiliary Code	It is controlled by Montage-LZ
FPK	Flash protection key	Usually it is from NAGRA, and used to encrypt REE bl2 image.
PIP	Picture in picture	one main play and one sub play

Reference

1. How to use the package

1.1 Prepare the environment：

Reference environment：

o Ubuntu 16.04.2 LTS

o GNU bash, version 4.3.48(1)-release-(x86_64-pc-linux-gnu)

o OpenSSL 1.0.2g 1 Mar 2016

o Python2.7

o mkimage 2016.01+dfsg1-2ubuntu5

1.2 Get our Sym6 DDK Repository：

Please apply account from our technical support engineer, and then you can clone the DDK repository from our Gerrit server.

2 SDK brief introduction

2.1 SDK Structure：

```
|-- nagra_modules
|     |-- Makefile
|     |-- MT_NOCSAPIs          -------- > NOCS API for Nagra project.
```

```
|     `-- nagra_pre_integraton_test    --------- > A demo project, but there is no Nagra test libs.
|-- patch_nagra_sdk.sh
|-- product
|     |-- configs
|     `-- symphony6_tee_nagra3x_demo
`-- tee_bsp
      |-- all_pss.img                  --------- >First stage boot image (For NAND flash), boot your APP.
      |-- arm64
      |     |-- tee-os
```

Please note, there is no NAGRA pre-integration test libs in “nagra_modules/nagra_pre_integraton_test”, so the demo project will conpile failed. If you want to make the DDK compile success by command “make”, please remove all “make -C nagra_pre_integraton_test xxx” in file “nagra_modules/ Makefile”, and then you will get the NOCA API libs:

“$(MT_BUILD_DIR)/libmt_nocsapi_impl.a”

“$(MT_BUILD_DIR)/libmt_nocsapi_impl.so”

for example:

./buildroot/output/host/aarch64-buildroot-linux-gnu/sysroot/usr/lib/mt/libmt_nocsapi_impl.so

./buildroot/output/host/aarch64-buildroot-linux-gnu/sysroot/usr/lib/mt/libmt_nocsapi_impl.a

2.2 How to use release package

2.2.1 Patch it to our DDK

Step1：decompress package by command “tar -xvf NASC3.x_Linux_S6_Temp20250717.tar”

Step2：cd to NASC3.x_Linux_S6_Temp20250717 and run “./patch_nagra_sdk.sh xxxx”.

“xxx” is the path of our DDK, for example: ./patch_nagra_sdk.sh ../linux_sdk0/Lznux/ddk/

Step3：the script copy nagra_modules, tee_bsp and productto DDK now.

Please note, “product\configs\kernel_configs\ symphony6_nagra3x_demo_defconfig” and “product\configs\symphony6_tee_nagra3x_demo.cfg” are demo configure files based on DDK0519 now. If you want to use some other DDK versions (eg: DDK 0616), please just do some new configure based on your DDK. In fact, for NAGRA case you only should make your configure file (eg: product\configs\kernel_configs\symphony6_nagra3x_demo_defconfig) “”include “CONFIG_MT_MMZ_ISOLATION_DDR=y” and “CONFIG_MT_CERT=y”.

Please refer to below operation to enable them:

1), run command “make kernel_menuconfig”, and then find below configure selections and set it.

```
Symbol: MONTAGE_NAGRA [=y]
Type  : bool
Defined at arch/arm64/platforms/symphony6/Kconfig:26
  Prompt: Montage Nagra CA
  Depends on: ARCH_SYMPHONY6 [=y]
  Location:
    -> Platform selection
      -> Montage Symphon6 SOC platform (ARCH_SYMPHONY6 [=y])
(1)     -> Montage Nagra CA (MONTAGE_NAGRA [=y])
```

```
Symbol: MT_MMZ_ISOLATION_DDR [=y]
Type  : bool
Defined at mm/Kconfig:927
  Prompt: Set the DDR MMZ isolate from the buddy system
  Depends on: MT_MMZ_ISOLATION_AV [=y]
  Location:
    -> Memory Management options
      -> Contiguous Memory Allocator (CMA [=y])
        -> Set the AV && Audio MMZ isolate from the buddy system (MT_MMZ_ISOLATION_AV [=y])
(4)       -> Set the DDR MMZ isolate from the buddy system (MT_MMZ_ISOLATION_DDR [=y])
```

2.2.2 Demo project

There is a demo project named “symphony6_tee_nagra3x_demo”. You can modify it to your project.
Project path：ddk/product/symphony6_tee_nagra3x_demo
Project configure path：linux/product/configs/ symphony6_tee_nagra3x_demo.cfg
Project kernel configure path：linux/product/configs/kernel_configs/ symphony6_nagra3x_demo_defconfig

How to compile?

source envsetup symphony6_tee_nagra3x_demo.cfg

make clean;make

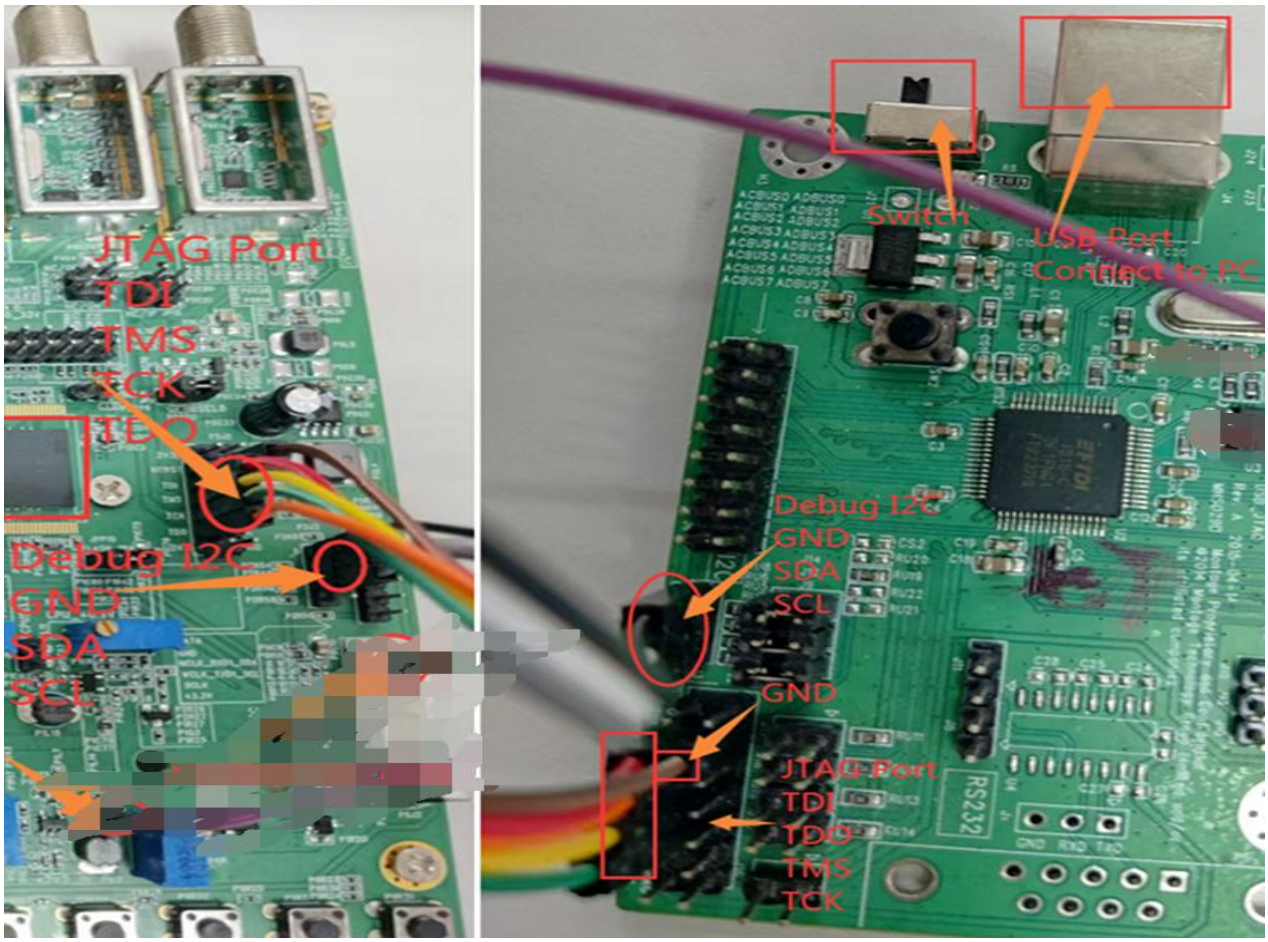
2.3 Flash program

Please note: Before use JTAG program flash image, please use debug I2C to connect the board and configure UART parameters by running commands “catch_fail” and “batch uart.txt” in debug I2C window.

After decompress the package “sym6_i2c_jtag.7z”, please copy the file “uart.txt” to it.

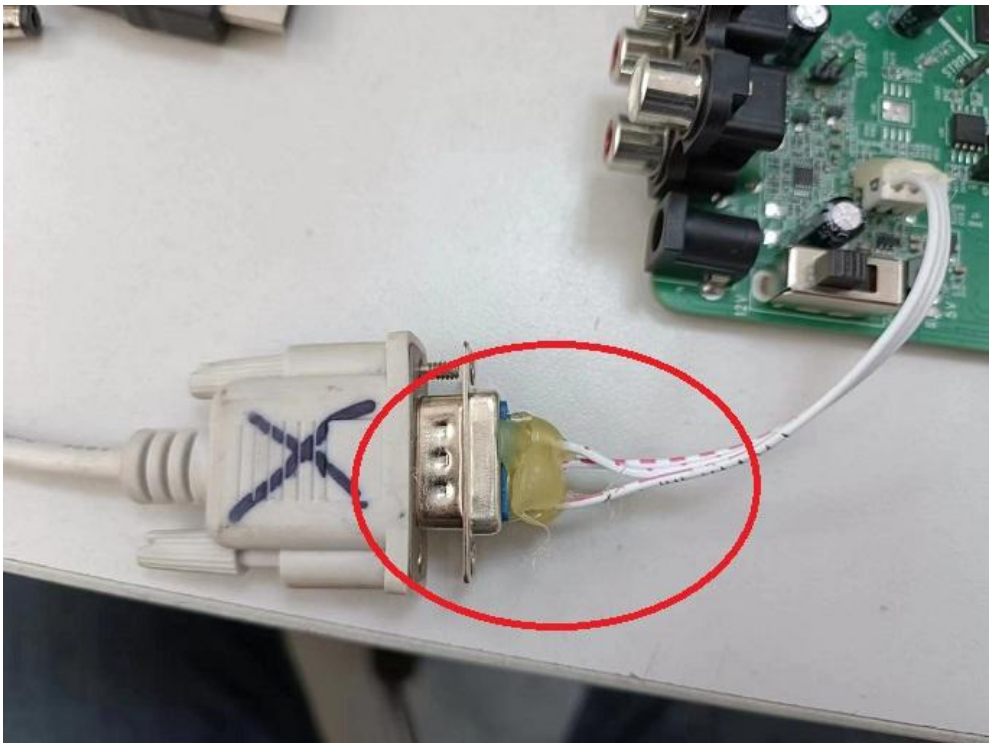
2.3.1 Setup hardware environment

Step 1: connect JTAG port to PC USB port and our JTAG board.

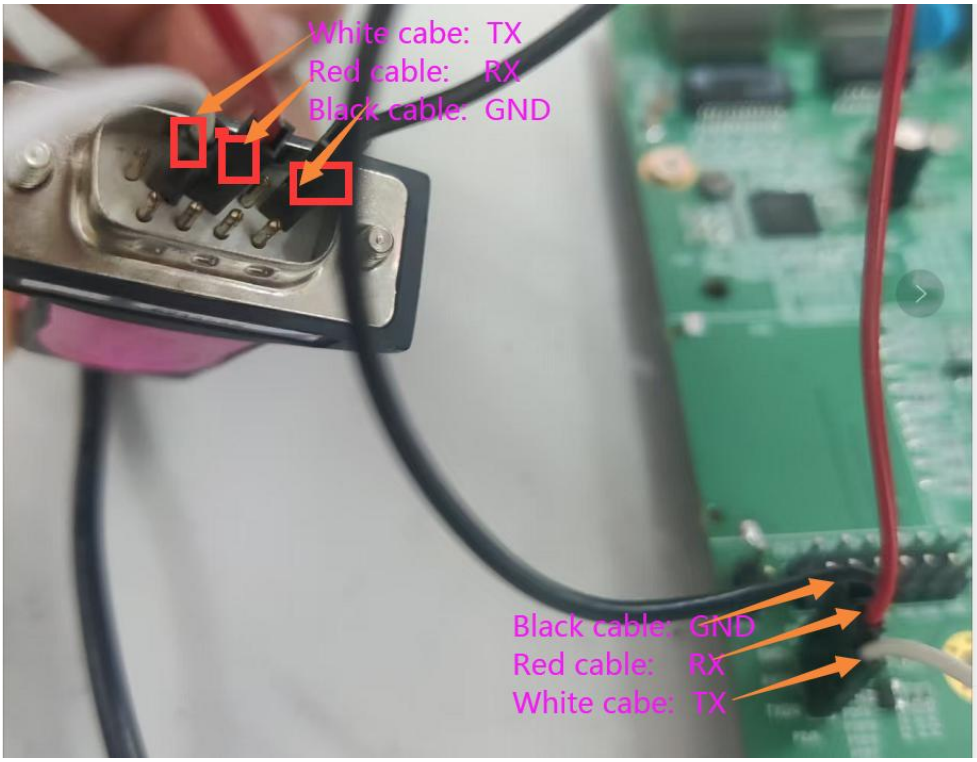


Debug I2C and JTAG connection

Step 2: Connect PC serial port to our test boar's UART, or connect the PC to our test board by USB Serial Converter.

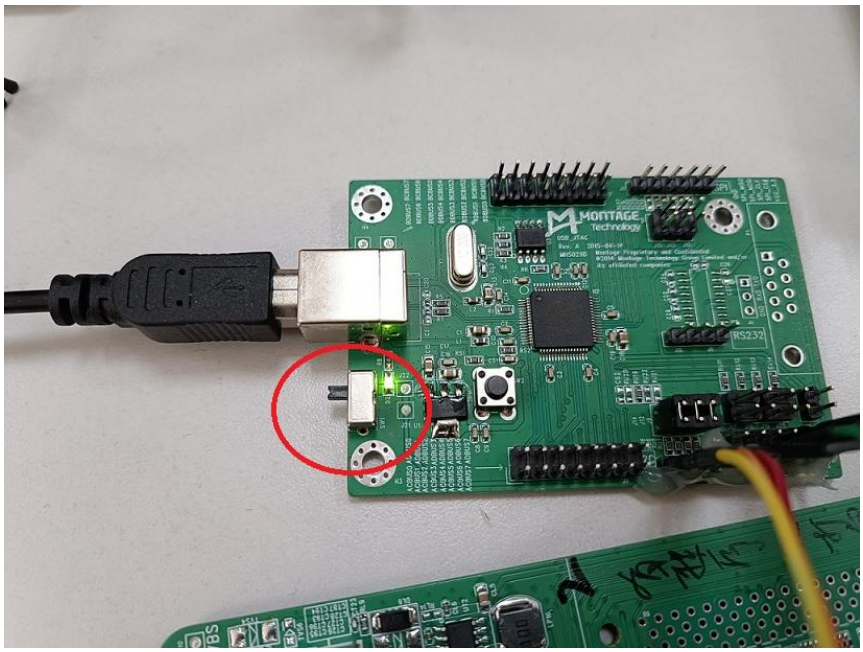


UART connector



UART connector: USB Serial Converter

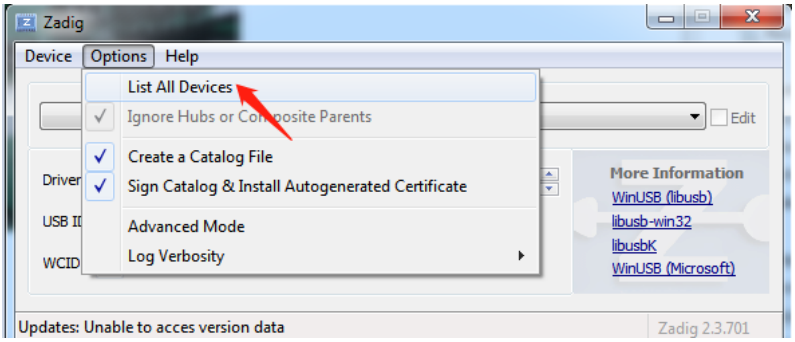
Step 3: Power on our JTAG board.



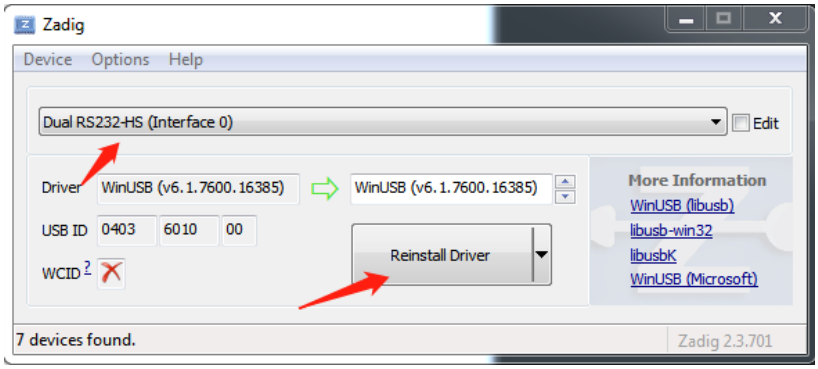
Step 4: Setup driver for our JTAG board。

a. Find “zadig-2.3.exe.7z” in our FIG tool, decompress and run “zadig-2.3.exe”.

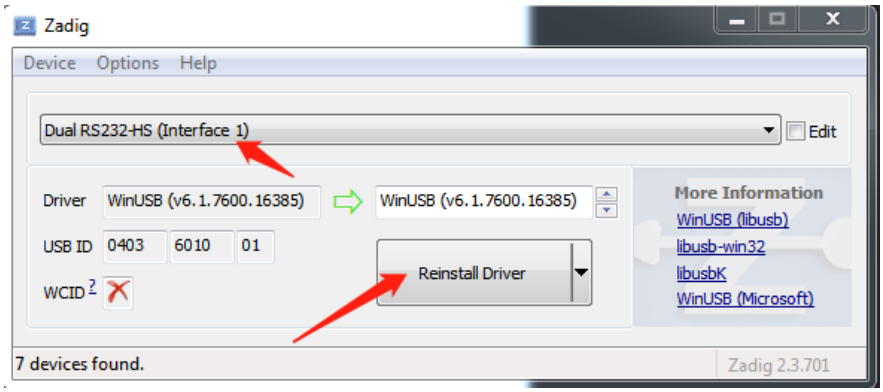
b. click “Options”, select “List All Devices”.



c. select “Dual RS232-HS(Interface 0)” and then run “Install Drive” or “Resintall Driver”



d. select “Dual RS232-HS(Interface 1)”, and then run “Install Drive” or “Resintall Driver”.

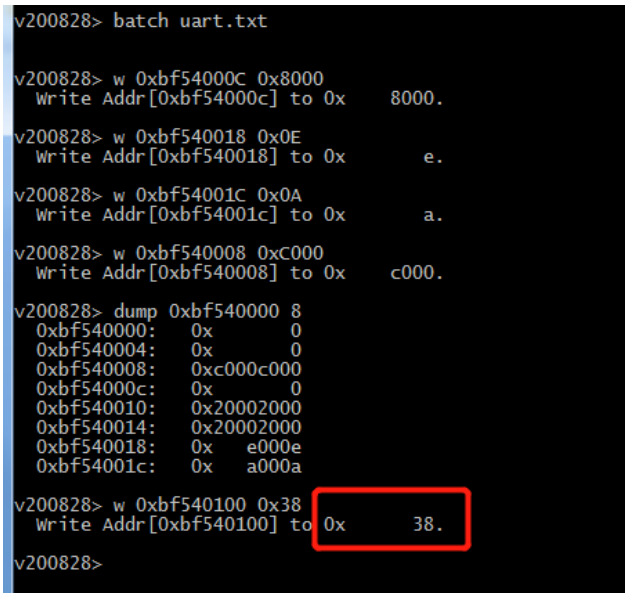


2.3.2 Flash program

Step 1: Power on the board and JTAG board first, then open debug i2c “dbg_i2c.exe”, and run “catch_fail”. If you get “success”, it means debug I2C connection well.

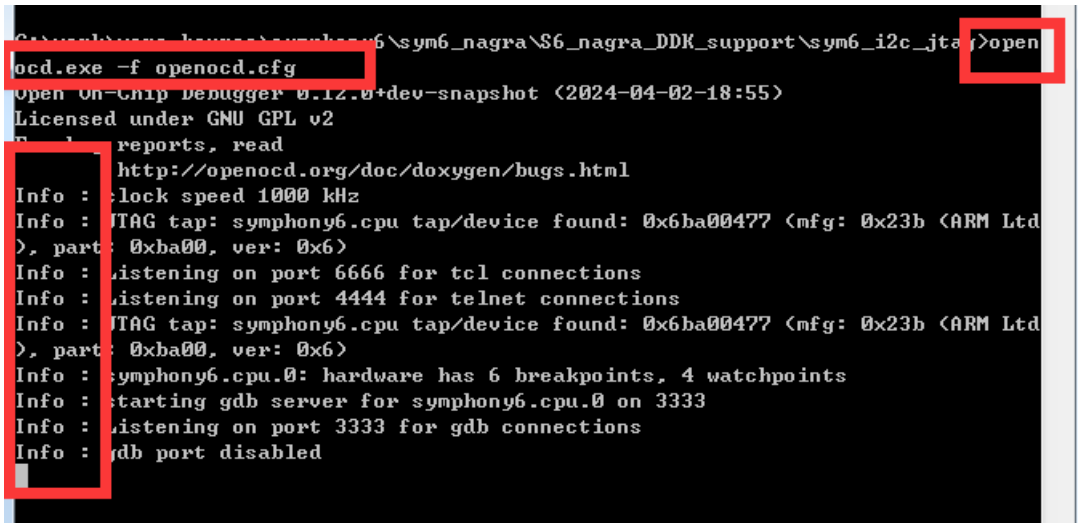
```
v200828> catch_fail

v200828> catch_fail
  catch(1 times) success......
v200828>
```

Step 2: Run command “batch uart.txt” to check if you can get the char ‘8’ in your PC serial tool window.



Step 3: Close debug I2C window, and open window command window, and then run command "flash_burn_S6_32.bat xxx.img 0" or "flash_burn_S6_64.bat xxx.img 0". ("flash_burn_S6_32.bat" is for 32 bits windows system, and "flash_burn_S6_64.bat" is for 64 bits windows system)

Please note: Before doing it, you can check if JTAG connects successfully first, by command "openocd.exe -f openocd.cfg". If there is no any error after "info:", it means JTAG connection success.



Step 4: Program flash success information.

Please note if there is something wrong about your program image which can't boot the chip successfully, you need disconnect the flash, before do step1 and step2. If doing step1 and step 2 successfully, you should connect the flash, and then do step3 and step4.

2.3.2.1 Flash write command

flash_burn_S6_32.bat <image path> <flash offset>
Write SPI-NAND, and PNAND flash for example：
flash_burn_S6_32.bat d:/work/all_pss.img 0
flash_burn_S6_32:/work/xxx.bin 0x800000

2.3.2.2 Flash read command

flash_read_S6_32.bat <image path> <flash offset> <size>
flash_read_S6_32.bat is for 32 bits system, and flash_read_S6_64.bat is for 64 bits system
Write SPI-NAND, and PNAND flash for example：
flash_read.bat d:/work/xxx.bin 0x800000 32
If you need to write more than one image, please run flash write command one by one.
Please Note:

1, above flash scripts are for SPI-NOR, SPI-NAND, and PNAND flash. They don't support EMMC, so if you want to use EMMC, please use scripts flash_burn_S6_64_emmc.bat , flash_read_S6_64_emmc.bat , flash_burn_S6_32_emmc.bat and flash_read_S6_32_emmc.bat. (32 is for 32bit system, 64 is for 64 bits system).

2, the flash structure is different from NOR and EMMC to other SPI-NAND, PNAND. So if you want to use EMMC, please program demo "all_emmc.img".

2.3.2.3 Update flash data by debug Boot

Sometimes we want to use debug boot to debug Linux system, and debug boot supports program flash data directly, so we can use it to update each flash images.

For SPI-NNAD, please refer to "update_spinand_tee_auto.vbs" in folder "ddk\product\chicago_tee". Please replace boot image xxx"xall.img" in script to NAGRA boot image "all_pss.img"

For PNAND, please refer to "update_rawnand_3byte_2k_tee_auto.vbs" in folder "ddk\product\chicago_tee". Please replace boot image xxx"xall.img" in script to NAGRA boot image "all_pss.img".

For EMMC, please refer to "update_emmc_boot_part.vbs" （booting pa artition） and update_emmc_tee_auto.vbs （other partition） in folder "ddk\product\chicago_tee". Please replace boot image xxx"xall.img" in script to NAGRA boot image "all_emmc.img"

3 NOCS3.x chipset introduce

Please refer to <<Symphony6_NOCS3.x_Chipset_Specific_Configuration_for_STBMs_v1.2>> for detail.

4 Bootloader

Please refer to related documents: for example “NASC3.2 高安 Boot 开发指南_v1.0.0.docx”

5 TEE related content (NASC3.x Dual Only)

5.1 TA files

If you get any TA files, please copy them to our DDK path "tee_bsp/arm64/usr/lib/optee_armtz".
NAGRA TA files:
bc2f95bc-14b6-4445-a43c-a1796e7cac31.root.cert ----> From Montage-LZ (need give TEE MSID to Montage-LZ first)
bc2f95bc-14b6-4445-a43c-a1796e7cac31.cert ----> From NAGRA
bc2f95bc-14b6-4445-a43c-a1796e7cac31.ta ---->From NAGRA

Please note: bc2f95bc-14b6-4445-a43c-a1796e7cac31.root.cert is from montage-lz. Before you use NAGRA TA, please program TA segment ID (OTP_CA_TA_MSID, offset byte address 0x39FC, bit address 0x39FC*8) to OTP. Usually the test TA segment ID is 0x00000005, but the ID is different in NAGRA production TA file, so when you use NAGRA production TA, please check the correct TA segment ID first! The NAGRA TA files in chipset NASC3.x SDK patch package are test version which cannot be used in your project. Please get and use production TA files from NAGRA.

```
MT_U32 ta_msid = 0;
MT_UNF_OTP_read(0xE7F * 32, 32, &ta_msid);
printf("TA SegmentID:0x%04x\n", ta_msid);
if (ta_msid != 0x0005) {
            MT_UNF_OTP_write(0xE7F * 32, 32, 0x0005);
}
```

Note:
There may be some other TA files according to your project functions. And normally the MSID and version of them are 0x00000000, but Montage-LZ has the right to modify the relevant MSID and Version, so the specific information needs to be confirmed with montage before using them.

1. Audio TA files: There are several different versions for different function, and Montage-LZ releases them according to your requirement. So the TA files are not in chipset NASC3.x SDK patch package. For example whether to support Dolby:
   ea38a79f-b962-487c-86d3-b3d1d94383f9.root.cert

ea38a79f-b962-487c-86d3-b3d1d94383f9.cert
ea38a79f-b962-487c-86d3-b3d1d94383f9.ta

2. NexGuard TA: It is form Montage-LZ, and it is for Nagra NexGuard watermark function only. Usually it is released in chipset NASC3.x SDK patch package.
   dee067b0-0834-4c86-b99e-09f00fd1329c.root.cert
   dee067b0-0834-4c86-b99e-09f00fd1329c.cert
   dee067b0-0834-4c86-b99e-09f00fd1329c.ta

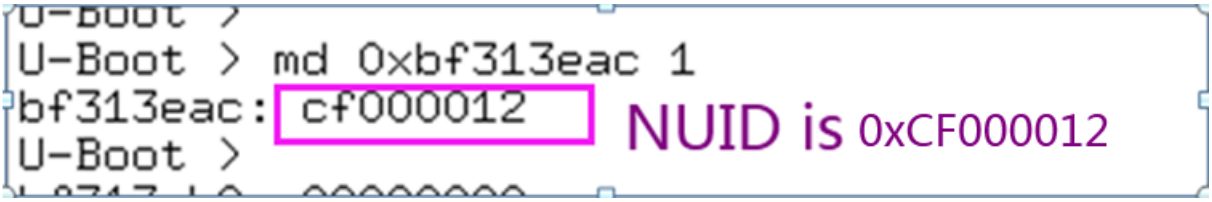
   Please note: NexGuard Watermark can be controlled both in REE and TEE side. In TEE side it is used by NAGRA TA, but in REE side it is controlled by special API, you can reference the test implement in “nagra_modules/nagra_pre_integraton_test/ngwm_ree.c”.

3. When you have problem about running NAGRA test TA files, please check below one by one:

a) Check Nagra TA segment ID. (Usually it is 0x5 for test TA). For test, you can just run below command in Uboot:

```
mw.l 0xbf314400 0x33250792    //Open the write right
mw.l 0xbf3139fc 0x00000005    // Write Nagra TA segment ID to OTP filed.
md 0xbf3139fc 1  // Read the TA segment ID from OTP field, and it should be 0x05
```

b) Get CSC and PK data from NAGRA by your chipset NUID. (Run command “md.l 0xbf313eac 1” in Uboot to get chipset NUID)



c) Get the CASN from Nagra and program it to OTP. For test, you can just run below command in Uboot:

```
mw.l 0xbf314400 0x33250792    //Open the write right
mw.l 0xbf313e74 0x2C926    // Write NAGRA CASN to OTP filed. 0x2C926 is a demo value, please check the right value with NAGRA!
md 0xbf313e74 1            // Read to check if the value is right
```

d) Get test TA files form NAGRA and Montage-lz.

bc2f95bc-14b6-4445-a43c-a1796e7cac31.root.cert is from Montage and please get it under folder “Nagra_testTA_ProductionKey”. bc2f95bc-14b6-4445-a43c-a1796e7cac31.cert and bc2f95bc-14b6-4445-a43c-a1796e7cac31.ta are from NAGRA.

Please note, if you want to update the NAGRA TA files to test boards, please make sure you remove “/data/tee” or re-burn all Linux file system firstly.

4. Montage Auxiliary TA

In some special scenario, when a program transitions from clear to scramble, and NAGRA TA activates SMP, there may be a momentary snow screen issue. So we add Auxiliary TA to first enable the SMP settings, thereby achieving the goal of optimizing and eliminating the issue. But if you don't care about this special scenario, please ignore related operation and below TA files.

64eed1a2-de8d-ff79-0ec7-db50ce3632bf.root.cert
64eed1a2-de8d-ff79-0ec7-db50ce3632bf.cert
64eed1a2-de8d-ff79-0ec7-db50ce3632bf.ta

There are several APIs about auxiliary TA in REE side.

Related APIs in REE side:

```
/* please call it after you do Nagra TA init */
mt_u32 mtExtMTLZ_TEECInit(void)

/* please call it after you do NAGRA TA terminate */
mt_u32 mtExtMTLZ_TEECDeinit(void)

/* please call it when you open NAGRA session of play program */
mt_u32 mtExtMTLZ_TEE_SMP_En(bool isMianPlay, MTPlayType mt_play)

/* please call it after you close NAGRA session of play program */
mt_u32 mtExtMTLZ_TEE_SMP_Dis(bool isMianPlay)
```

5.2 TEE-OS files

TEE-OS related files are "tee_bsp/arm64/tee-os/512/tee-os.bin" and "tee_bsp/arm64/tee-os/512/tee.dtb.bin". They both support 512M and 1024M DDR solution.
TEE OS will read and write "/data/tee", so there should have "/data" folder, and "/data" folder should can be read and write right.

Please note, if you want to update the NAGRA TA files to test boards, please make sure you remove "/data/tee" or re-burn all Linux file system firstly.

6 OTP brief

There are two type OTP fields need STBMs to pay attention. Some OTP fields are operated by NAGRA API, such as market segment Id,CA_SN, REE secure enable…; other OTP fields needs STBMs program them according to project, these OTP are in section 5 "OTP Specific Configuration for STBMs " of doc "Symphony6_NOCS3.x_Chipset_Specific_Configuration_for_STBMs".

If there is any question about OTP fields, please ask Montage-LZ for support.

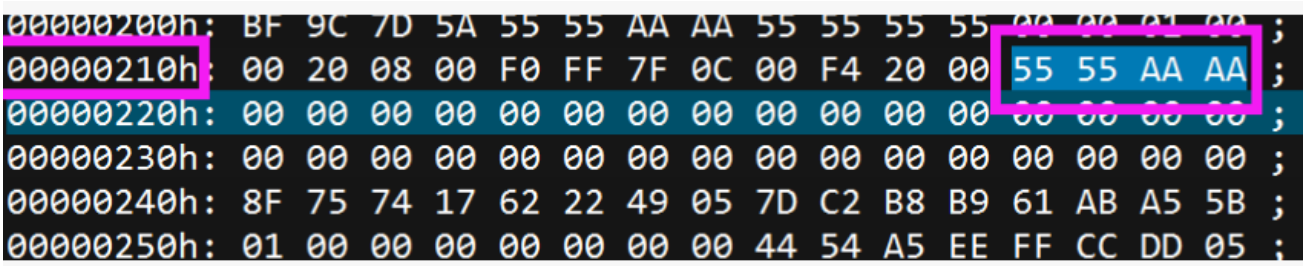
7 Get singed SCS image and program to board.

7.1 Get singed SCS image from NAGRA

After you using our flash tool to generate a SCS image “all.img”, you can use it to develop you boot flow. But for final production board, you should use signed SCS image which is from NAGRA. So you should generate an image and send it to NAGRA to get a signed and encryption one.

Please note, you should preprocess your test SCS image “all.img” before sending it to NAGRA.

a) Please modify the data to “5555AAAA” from offset 0x21C. (You can refer to our script “generate_for_sign.sh” in flash tool package).

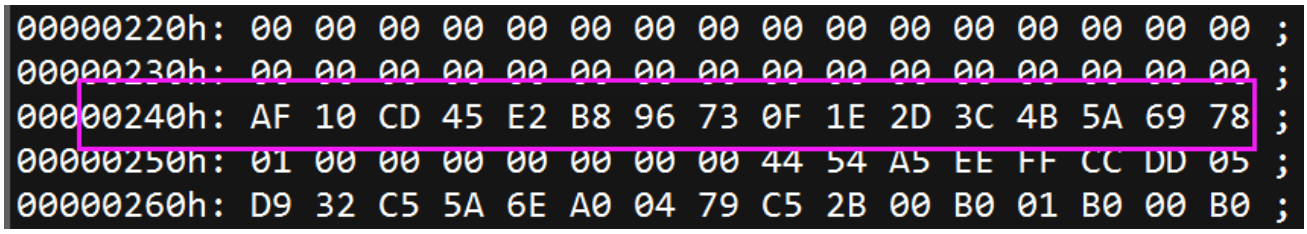


b) Although you can configure “dte_boot_code_total_size” in nagra_bsd_demo.ini under our flash tool package different value. (One from 0x40000, 0x80000, 0x100000, 0x200000). But NAGRA requires that a image of a specific length must be used to apply for a signature. For SPI-NOR and EMMC case the length must be 0x220000, and for SPI-NAND and P-NAND case the length must be 0x280000. So please padding you image to requested length with 0x00 or 0xFF according to your boot mode.

c) After you get the signed SCS image “xxx_signed.img” from NAGRA, you can only keep the length you need, and the padded part can be removed. For SPI-NOR and EMMC case the length can be one of 0x60000, 0xA0000, 0x120000 and 0x220000 and for SPI-NAND and P-NAND case the length can be one of 0xC0000, 0x100000, 0x180000 and 0x280000.
Please note: Usually NAGRA will encrypted the bl2 image data in signed SCS image. And you should get REE segment ID and FPK (flash protection key) from NAGRA too.

7.2 Program singed SCS image to board

After you get the signed SCS image from NAGRA and check the real length you wanted, now you can program it to you board. But firstly, you should encrypt FPK by NAGRA API “TSecEncryptFlashProtKey” (in APP) or “bsdEncryptFlashProtKey” (in REE BL2 or BL3) and then program it to offset 0x240 of flash or EMMC.



Please note: NAND flashs (both SPI-NAND and PNAND) have bad bock management. We use offset address 0x20000 ~0x80000 to support it. So you should divide the SCS signed image to two parts, and program them to NAND flash one by one. First part is from 0 to 0x20000, and programs it to flash offset 0x0 to 0x20000. Second part is for 0x80000 to the end of valid data (0xC0000, 0x100000, 0x180000 or 0x280000), and programs it to flash offset 0x80000. And we recommend that you also reserve sufficient bad blocks for the second part, for example 0x100000.

7.3 Test security boot of SCS image.

1) Before enable SCS flag “OTP_SECURE_EN” in OTP filed (from bit4 to bit15 of OTP offset byte address 0x3A3C, bit address 0x3A3C * 8) by NAGRA API or our OTP write API, you should program Nagra REE segment ID (NAGR API xxxSetMarketSegmentId) and version firstly. Usually version is 0x0, so you don’t need to program it. The OTP field of segment ID is “OTP_CA_MSID” (from bit0 to bit32 of OTP offset byte address 0x30F4, bit address 0x39F4 * 8). The OTP field of version is “OTP_CA_Version_REF” (from bit0 to bit32 of OTP offset byte address 0x30F8, bit address 0x39F48* 8).

2) After programming correct TEE TA MSID and version, you can modify 4 bytes of offset address 0xF3C0 to non-zero data, for example 0x33333333, to verify if the SCS image can boot well. It means you can secure boot flow before enable SCS flag “OTP_SECURE_EN” in OTP filed. It is a very kindly debugging method for STBMs.

3) If step 2 is work well, it means your production flow is well for SCS image. So please use the final image and enable SCS flag “OTP_SECURE_EN” in OTP filed.

4) Please also make sure you program “csc” and “pk” data correctly, otherwise you will have problem about do NAGRA CAK work. “csc” and “pk” are from NAGRA, and please refer to “nagra boot” for detail about how to encrypt and program them.

8 Brief flow of Montage-LZ case

Here just introduce Montage-LZ’s CA API about NAGRA case. Any about NAGTA CA flow and APIs, please refer to NAGRA documents.

There are some demo codes under “nagra_modules\nagra_pre_integraton_test”. It is not a complete project, and it is not necessary for your project, but it can help you to understand our software in NAGRA case. And the code under “nagra_modules\MT_NOCSAPIs\src” are the necessary APIs implement for NAGRA requirement or your project.

8.1How to boot NAGRA TA

NAGRA TA can be open and close dynamic in our software system, so please init and terminate it according to below code before booting it.

```
#include <tee_client_api.h>
#include "nvta_tflts_gptee.h"

#define TFL_DYNC_SHMEM_SIZE (3 * 1024 * 1024)

uint8_t tfl_dync_sh_mem[TFL_DYNC_SHMEM_SIZE] = {1};
TEEC_SharedMemory tfl_shm = {
    .buffer = (void *)tfl_dync_sh_mem,
    .size = TFL_DYNC_SHMEM_SIZE,
    .flags = TEEC_MEM_INPUT | TEEC_MEM_OUTPUT,
};

TEEC_Result tee_res;
TEEC_Context g_teec_ctx;
```

Init code:

```
tee_res = TEEC_InitializeContext(NULL, &g_teec_ctx);
if (tee_res != TEEC_SUCCESS) {
        EMSG("Init tee context failed\n");
}
/*store the context*/
nvGpTeeConfigure(&g_teec_ctx);

tee_res = TEEC_RegisterSharedMemory(&g_teec_ctx, &tfl_shm);
if (tee_res != TEEC_SUCCESS) {
        EMSG("Register dynamic share memory failed, ret: 0x%08x\n", tee_res);
        nvGpTeeConfigure(NULL);
        TEEC_FinalizeContext(&g_teec_ctx);
}
```

Terminate code:

```
nvGpTeeConfigure(NULL);
TEEC_ReleaseSharedMemory(&tfl_shm);
TEEC_FinalizeContext(&g_teec_ctx);
```

Please note: maybe there are some modules clock is disabled, so in order to ensure that the NAGRA's CERT and various algorithms work properly, please call API "mtSecRelatedModuleClkEnable" first. The API will enable the clocks of the relevant modules.

8.2 How to do DVB/PVR/OTT operation

Under "nagra_modules\nagra_pre_integraton_test", there are some demo code files which are implemented according to NAGRA pre-integration test environment.

1 DVB play:

Please refer to "nvSprOpenDvbSession" and "nvSprCloseSession" in "nv_spr.c" file. And TS stream is injected to DMX by CPU. You project may different from it. But you should notice below API calling: "nvSprOpenDvbSession": It is a Nagra test API, not used in your project.

a) Please manage "tsid" by yourself, and it should be from 1 to 31; it is used by our APIs to different resource for services. Usually DVB uses 1-19, and OTT uses 20-25.

b) Calling “mtSecGetPlatDmxId” to get useful DMX ID.

c) If your project supports PIP, please check current play is main video or sub video. If it is sub video, please set “mtSecSetIsMultiSessionFlag”. And main video or sub video use different DDR ranges; please configure the DDR ranges by “mtSecSetProtectBuffer” with different type “MB_SUB_VID”, “MB_AUD” or “MB_VID”. The DDR ranges of protected buffer for both main video and sub-video should be consistent with the fixed value configured in your boot arguments, but The DDR ranges of protected buffer for audio should be dynamically allocated by Montage-LZ driver.

Please refer to below demo configuration, if you want to do any modification, please confirm with Montage-LZ first. (Here, we don’t include FTA common operation.)

Main screen:

```
/* Set Video buffer to be fixed range */
#define MM_AV_MMZ_START                 0xFE00000//254M
#define MM_AV_MMZ_SIZE                  0xBC00000//188M

mt_handle audioChannel;

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, FALSE);

/* Set Vedio buffer for SMP*/
mtSecSetProtectBuffer(MM_AV_MMZ_START, MM_AV_MMZ_SIZE, MB_VID);
/* Get current audio PID channel*/
MT_UNF_DMX_GetChannelHandle(ts->playDmxId, audPid, &audioChannel);
/* Get the buffer information by current audio PID channel*/
MT_UNF_DMX_GetEsBuffPhyAddr(audioChannel, (ulong *)&esAudioBuffPhyAddr, &esBuffSize);
/* Set audio buffer for SMP */
mtSecSetProtectBuffer(esAudioBuffPhyAddr, esBuffSize, MB_AUD);
/* set the main screen TSID for nexgurd watermark */
mt_ngwm_configure_mainID(tsid);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

Sub screen (PIP function):

```
#define MM_PIP_ZONEMMZ_START    0x1BA00000
#define MM_PIP_ZONEMMZ_SIZE       0x3E00000

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, TRUE);
/* Set sub-viedo buffer for SMP */
mtSecSetProtectBuffer(MM_PIP_ZONEMMZ_START, MM_PIP_ZONEMMZ_SIZE, MB_SUB_VID);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

d) Bind current play PID list to “tsid” by calling “mtSecSetSessionPid”. (If there are more than one audio channel, please just using current audio PID)

2 PVR

PVR includes record and replay operations.

As long as doing the PVR operation please set the relevant protect buffer again. When doing PVR record, since the DMX port and record channel can be freely paired, it is necessary to perform a binding operation between the DMX ID used for soft injection and the record channel.

Record:

```
/* Set Video buffer to be fixed range */
#define MM_AV_MMZ_START                 0xFE00000//254M
#define MM_AV_MMZ_SIZE                  0xBC00000//188M

/* Get current platform demux ID */
ts->recordDmxId = mtSecGetPlatDmxId(tsid)
……
//Attach DMX port
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, FALSE);

/* Set Vedio buffer for SMP*/
mtSecSetProtectBuffer(MM_AV_MMZ_START, MM_AV_MMZ_SIZE, MB_VID);
/* Get current audio PID channel*/
MT_UNF_DMX_GetChannelHandle(ts->playDmxId, audPid, &audioChannel);
/* Get the buffer information by current audio PID channel*/
MT_UNF_DMX_GetEsBuffPhyAddr(audioChannel, (ulong *)&esAudioBuffPhyAddr, &esBuffSize);
/* create Montage-LZ's PVR record functions */
MT_UNF_PVR_RecInit();
/* Register PVR callback for events, and set protected buffer in callback it for SMP */
MT_UNF_PVR_RegisterEvent(MT_UNF_PVR_EVENT_REC_DMX_CREATE, MTADP_PVR_CallBack, NULL);
/* Set if current record is first one? */
mtSecSetIsMultiSessionFlag(ts->tsid, record_pip_flg);
/* Set record video PID */
MT_UNF_PVR_RecSetPid(ts->recordChannelId, pvrRecordAttr.u32DemuxID, (int)vidPid);
/* Set record audio PID */
MT_UNF_PVR_RecSetPid(ts->recordChannelId, pvrRecordAttr.u32DemuxID, (int)audPid);
/* Register callback function for PVR, and you can encrypt data in it */
MT_UNF_PVR_RegisterExtraCallback(ts->recordChannelId, MT_UNF_PVR_EXTRA_WRITE_CALLBACK,
                                                recordingWriteCallback, NULL);
/* Start PVR */
MT_UNF_PVR_RecStartChn(ts->recordChannelId);
……
……
/* Set current PVR PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

Event callback for PVR Record:

```
mt_void MTADP_PVR_CallBack(mt_u32 u32ChnID, MT_UNF_PVR_EVENT_E EventType, mt_s32
s32EventValue, mt_void *args) {
        /* Record chnnel, 0-3 is valid, so please set 0xFF as default value*/
        mt_u8 pvrRecodChan = 0xFF;
        /* Get buffer information of current PVR event */
        MT_UNF_DMX_GetRecBufferStatus((mt_handle)s32EventValue, &stStatus);
        /* Get low 8 bits data of callback event value as record channel ID */
        pvrRecodChan = s32EventValue & 0xFF;
        /* only 0-3 are valid record channel IDs */
        if (pvrRecodChan <= 3)
        {
                /* Bind record channel ID and DMX port, DMX port is got when calling
MT_UNF_DMX_CreateTSBuffer */
                mtSecSetRecChanByDmxID(MT_UNF_DMX_PORT_RAMx, pvrRecodChan);
    } else {
        /*It is invalid record channel*/
        Return;
    }
        /* Select one from "MB_REC/ MB_REC_1/ MB_REC_2/ MB_REC_3" according to
MT_UNF_DMX_PORT_RAM_x */
        mtSecSetProtectBuffer(mb.start, mb.size, MB_REC);
        or mtSecSetProtectBuffer(mb.start, mb.size, MB_REC_1);
        or mtSecSetProtectBuffer(mb.start, mb.size, MB_REC_2);
        or mtSecSetProtectBuffer(mb.start, mb.size, MB_REC_3);
}
```

Replay in Main screen:

```
/* Set Video buffer to be fixed range */
#define MM_AV_MMZ_START                 0xFE00000//254M
#define MM_AV_MMZ_SIZE                  0xBC00000//188M

mt_handle audioChannel;

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, FALSE);

/* Get TS buffer with 256K algined */
MT_UNF_DMX_GetTSBufferEx
/* Set the buffer is OTT for replay */
mtSecSetProtectBuffer(phyAddrTsBuf, DEMUX_TS_BUF_SIZE, MB_OTT);
/* Set Vedio buffer for SMP*/
mtSecSetProtectBuffer(MM_AV_MMZ_START, MM_AV_MMZ_SIZE, MB_VID);
/* Get current audio PID channel*/
MT_UNF_DMX_GetChannelHandle(ts->playDmxId, audPid, &audioChannel);
/* Get the buffer information by current audio PID channel*/
MT_UNF_DMX_GetEsBuffPhyAddr(audioChannel, (ulong *)&esAudioBuffPhyAddr, &esBuffSize);
/* Set audio buffer for SMP */
mtSecSetProtectBuffer(esAudioBuffPhyAddr, esBuffSize, MB_AUD);
/* set the main screen TSID for nexgurd watermark */
mt_ngwm_configure_mainID(tsid);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

Replay in Sub screen (PIP function):

```
#define MM_PIP_ZONEMMZ_START    0x1BA00000
#define MM_PIP_ZONEMMZ_SIZE       0x3E00000

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Get TS buffer with 256K algined */
MT_UNF_DMX_GetTSBufferEx
/* Set the buffer is OTT for replay */
mtSecSetProtectBuffer(phyAddrTsBuf, DEMUX_TS_BUF_SIZE, MB_OTT);
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, TRUE);
/* Set sub-viedo buffer for SMP */
mtSecSetProtectBuffer(MM_PIP_ZONEMMZ_START, MM_PIP_ZONEMMZ_SIZE, MB_SUB_VID);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

3 OTT

Usually OTT includes HLS mode and DASH mode. For HLS mode, it is similar as PVR replay, so please refer to it. The

OTT in Main screen:

```
/* Set Video buffer to be fixed range */
#define MM_AV_MMZ_START                 0xFE00000//254M
#define MM_AV_MMZ_SIZE                  0xBC00000//188M

#define MM_OTT_START                    0xAA00000//170M boot arg ott0 buffer
#define MM_OTT_SIZE                             0x1000000// 16M

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, FALSE);
/* Set OTT buffer for SMP, you can use fixed PVR0 bufffer or malloced dynamic */
mtSecSetProtectBuffer(MM_OTT_START, MM_OTT_SIZE, MB_OTT);
/* Set Vedio buffer for SMP*/
mtSecSetProtectBuffer(MM_AV_MMZ_START, MM_AV_MMZ_SIZE, MB_VID);
/* Get current audio PID channel*/
MT_UNF_DMX_GetChannelHandle(ts->playDmxId, audPid, &audioChannel);
/* Get the buffer information by current audio PID channel*/
MT_UNF_DMX_GetEsBuffPhyAddr(audioChannel, (ulong *)&esAudioBuffPhyAddr, &esBuffSize);
/* Set audio buffer for SMP */
mtSecSetProtectBuffer(esAudioBuffPhyAddr, esBuffSize, MB_AUD);
/* set the main screen TSID for nexgurd watermark */
mt_ngwm_configure_mainID(tsid);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

OTT in Sub screen (PIP function):

```
#define MM_PIP_ZONEMMZ_START    0x1BA00000
#define MM_PIP_ZONEMMZ_SIZE       0x3E00000

/* Get current platform demux ID */
ts->playDmxId  = mtSecGetPlatDmxId(tsid)
……
……
/* Set main screen to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, TRUE);
/* Set OTT buffer for SMP, you can use fixed PVR0 bufffer or malloced dynamic */
mtSecSetProtectBuffer(MM_OTT_START, MM_OTT_SIZE, MB_OTT);
/* Set sub-viedo buffer for SMP */
mtSecSetProtectBuffer(MM_PIP_ZONEMMZ_START, MM_PIP_ZONEMMZ_SIZE, MB_SUB_VID);
……
……
/* Set current play PID to session with SMP always on */
mtSecSetSessionPid(tsid, &pxPidInfo, TRUE);
```

4 OTT re-encryption

Before use OTT re-encryption function, please set OTT protect buffer first. And it should be consistent with the fixed value configured in your boot arguments (eg: ott0). Our platform support 2 channels parallel operation. Please set “mtSecSetIsMultiSessionFlag” with “FALSE” and “TRUE” to different 2 channels.

OTT re-encryption in first channel

```
/* Set OTT buffer to be fixed range */
#define MM_OTT_START                    0xAA00000//170M boot arg ott0 buffer
#define MM_OTT_SIZE                             0x1000000// 16M

/* Set first channel to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, FALSE);
/* Set OTT buffer for SMP, you can use fixed ott0 area */
mtSecSetProtectBuffer(MM_OTT_START, MM_OTT_SIZE, MB_OTT);
…….
```

OTT re-encryption in second channel:

```
/* Set second channel to current session */
mtSecSetIsMultiSessionFlag(ts->tsid, TRUE);
/* Set OTT buffer for SMP, you can use fixed ott0 area */
mtSecSetProtectBuffer(MM_OTT_START, MM_OTT_SIZE, MB_OTT);
……
```

9 HDCP key encryption and loading

Now we support encrypt HDCP key of HDMI in NAGRA boot and test APP with new version TEE-loader and TEE-OS which are released after 2025-12-25.NAGRA boot stage. In APP stage, please refer to API “MT_UNF_HDCP_encrypt_hdcpkey_tee” to encrypt HDCP key and “MT_UNF_HDCP_load_hdcpkey” to loader HDCP key.

Please note:

1 If your DDK doesn’t support those APIs, please update “msp/inc/mt_unf_hdcp.h” and “msp/api/hdcp/mt_unf_hdcp.c”.

```
typedef struct mtUNF_HDCPKEY_HDCP_S
{
   MT_BOOL enc_flag; //if true, the input hdcpkey is encrypted.
   mt_u8 ext_encrypt_key[16]; //Ignore this, we use a default key to enc/dec.
   mt_u8 hdcpkey[304]; //input hdcpkey data, encrypted or clear, depends on enc_flag.
} MT_UNF_HDCP_HDCPKEY_S;
/**
\brief Encrypt HDCP key. HDCP key is encrypted using hdcp root key in otp.
\param[in]:st_hdcpkey.This parameter is used to define hdcp key in encrypted mode or clear mode.
CNcomment: 该参数为HDCP key的参数定义,包含加密/非加密的方式. CNend
\param[out]:out_encrypted_key. Output the encrypted hdcp key.   CNcomment: 输出的加密后的HDCP
KEY参数。 CNend
\retval MT_SUCCESS  Call this API succussful.   CNcomment:API系统调用成功。 CNend
\retval MT_FAILURE  Call this API fails.                CNcomment:API系统调用失败。 CNend
*/
mt_s32 MT_UNF_HDCP_encrypt_hdcpkey_tee(MT_UNF_HDCP_HDCPKEY_S st_hdcpkey, mt_u8
out_encrypted_key[304]);
/**
\brief Encrypt HDCP key. HDCP key is encrypted using hdcp root key in otp.
\param[in]:st_hdcpkey.This parameter is used to define hdcp key in encrypted mode or clear mode.
CNcomment: 该参数为HDCP key的参数定义,包含加密/非加密的方式. CNend
\param[out]:out_encrypted_key. Output the encrypted hdcp key.   CNcomment: 输出的加密后的HDCP
KEY参数。 CNend
\retval MT_SUCCESS  Call this API succussful.   CNcomment:API系统调用成功。 CNend
\retval MT_FAILURE  Call this API fails.                CNcomment:API系统调用失败。 CNend
*/
mt_s32 MT_UNF_HDCP_load_hdcpkey(mt_u8 in_encrypted_key[304]);
```

10 Output control

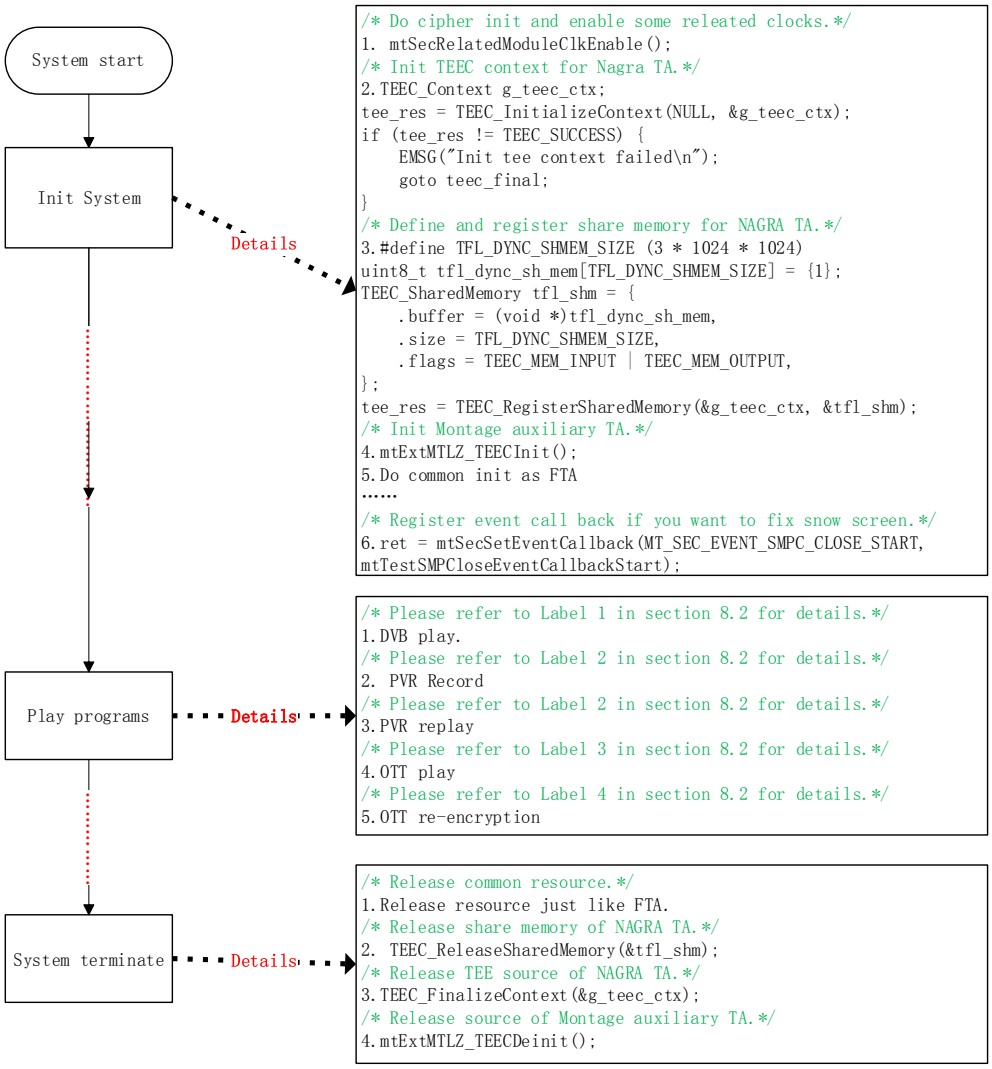
The usage rules of output control are parsed by NAGRA TA, and STBMs only need to check the status all the time (e.g. every 200ms) by our API “mtSecGetOpREEFlag”. If the API return true, it means some rules have been violated. Now you only need to check “HDCP1.x”, “HDCP2.0”, “4K”, “2K”, “HD”, “SD”, and the reference process code is in “nv_spr.c”.

Please note, if you don’t adjust resolution according to output control rules, the display device will not show program. But if your project doesn’t need to support downscale function, just ignore it.

```
typedef struct _MTSecOpREEFlag {
    TBoolean isViolation;  /** if there is any violation of output control.*/
    TUnsignedInt8 violateHDCP1;    /** HDCP 1.x violation.*/
    TUnsignedInt8 violateHDCP2;    /** HDCP 2.0 violation .*/
    TUnsignedInt8 violate4K;    /** 4K resolution violation: UHD.*/
    TUnsignedInt8 violate2K;    /** 2K/1K resolution violation: FHD.*/
    TUnsignedInt8 violateHD;    /** HD resolution violation: 720.*/
    TUnsignedInt8 violateSD;    /** SD resolution violation: less then 720.*/
} MTSecOpREEFlag;

MT_UNF_ENC_FMT_E encFormat = testFormat;
MTSecOpREEFlag opcinfor = {0x0,};
//Check if there is any violation.
if (mtSecGetOpREEFlag(&opcinfor) && opcinfor.isViolation)
{
        //If all of resolution are forbidden, there is no need to adjust the resolution.
        if ((opcinfor.violate4K && opcinfor.violate2K && opcinfor.violateHD &&
opcinfor.violateSD))
        {
            return;
        }
        //Check highest allowed resolution, and switch to it if it differs from the current one.
        if (opcinfor.violate4K == 0)
        {
            encFormat = testFormat; //" testFormat"  is the highest resolution of display device
        } else if (opcinfor.violate2K == 0)
        {
            encFormat = MT_UNF_ENC_FMT_1080i_50;
        } else if (opcinfor.violateHD == 0)
        {
            encFormat = MT_UNF_ENC_FMT_720P_60;
        } else if (opcinfor.violateSD == 0)
        {
            encFormat = MT_UNF_ENC_FMT_576P_50;
        }
        mtTestUpdateDisplayResolution(encFormat);// According to the project, implement
resolution switching
    }
```

11 Brief flow for NAGRA

Here, we only briefly introduce some our software flow for NAGRA CA, excluding the general process of FTA. Additionally, for the NAGRA APIs process, please refer to the NAGRA documentation.




```
/* Do cipher init and enable some releated clocks.*/
1. mtSecRelatedModuleClkEnable();
/* Init TEEC context for Nagra TA.*/
2.TEEC_Context g_teec_ctx;
tee_res = TEEC_InitializeContext(NULL, &g_teec_ctx);
if (tee_res != TEEC_SUCCESS) {
    EMSG("Init tee context failed\n");
    goto teec_final;
}
/* Define and register share memory for NAGRA TA.*/
3.#define TFL_DYNC_SHMEM_SIZE (3 * 1024 * 1024)
uint8_t tfl_dync_sh_mem[TFL_DYNC_SHMEM_SIZE] = {1};
TEEC_SharedMemory tfl_shm = {
    .buffer = (void *)tfl_dync_sh_mem,
    .size = TFL_DYNC_SHMEM_SIZE,
    .flags = TEEC_MEM_INPUT | TEEC_MEM_OUTPUT,
};
tee_res = TEEC_RegisterSharedMemory(&g_teec_ctx, &tfl_shm);
/* Init Montage auxiliary TA.*/
4.mtExtMTLZ_TEECInit();
5.Do common init as FTA
……
/* Register event call back if you want to fix snow screen.*/
6.ret = mtSecSetEventCallback(MT_SEC_EVENT_SMPC_CLOSE_START,
mtTestSMPCloseEventCallbackStart);
```

```
/* Please refer to Label 1 in section 8.2 for details.*/
1.DVB play.
/* Please refer to Label 2 in section 8.2 for details.*/
2. PVR Record
/* Please refer to Label 2 in section 8.2 for details.*/
3.PVR replay
/* Please refer to Label 3 in section 8.2 for details.*/
4.OTT play
/* Please refer to Label 4 in section 8.2 for details.*/
5.OTT re-encryption
```

```
/* Release common resource.*/
1.Release resource just like FTA.
/* Release share memory of NAGRA TA.*/
2. TEEC_ReleaseSharedMemory(&tfl_shm);
/* Release TEE source of NAGRA TA.*/
3.TEEC_FinalizeContext(&g_teec_ctx);
/* Release source of Montage auxiliary TA.*/
4.mtExtMTLZ_TEECDeinit();
```